

[L0-L1 Core Essence & Design Logic]

L0 Essence: The top-down network architecture bounds user space via sockets, utilizes sliding windows and dynamic congestion controls for reliable, high-throughput transport, routes packets via longest prefix matches and BGP policy routing, and resolves shared physical media contention using CSMA multiple-access protocols.

L1 Logic:

- 1. **Application Sockets & Resolution Boundary:** Applications submit data streams via a unified Socket file descriptor abstraction, while the DNS protocol runs cascading iterative queries to translate hostnames to IP addresses, providing destination coordinates.
- 2. **Sliding Windows & Retransmit Timers:** The transport layer manages packet flows via sliding windows to balance send rates and receive buffers, dynamically estimating RTTs to calculate safe timeouts (RTO) and trigger GBN or SR retransmissions on packet loss.
- 3. **Congestion Control Evolution (Cubic vs BBR):** Traditional Cubic relies on packet loss as a congestion indicator, scaling $cwnd$ via a cubic curve that fills link buffers and causes bufferbloat. BBR actively probes bottleneck bandwidth (BtlBw) and min RTT (RTprop) to run at the Kleinrock limits, achieving zero queue queuing.
- 4. **Hierarchical Autonomous Routing & Contention:** Networks use OSPF/Dijkstra inside Autonomous Systems (AS) and BGP attributes for inter-AS traffic engineering, while links resolve media access conflicts using CSMA/CD collision detection on wire and CSMA/CA collision avoidance on wireless channels.

[L2 Network Data Flow Topology]

- User write call (`write`) ☐ File Descriptor layer (`filewrite`) ☐ Pipe sync layer (`pipewrite`/wakeup reader) OR Inode cache (`write1`) ☐ Journal transaction layer (`write_log`) ☐ Buffer cache (`bwrite`) ☐ VirtIO Disk Driver (`virtio_disk_rw`) ☐ Hardware Disk Block.

[M1.1] HTTP/1.x Head-of-Line Blocking vs HTTP/2 Multiplexing (*HTTP/1.1* vs *HTTP/2*)

- **Head-of-Line Blocking (HOL):** HTTP/1.1 allows request pipelining but requires responses to be returned in strict request order. A slow response blocks all subsequent requests on the TCP connection.
- **Binary Framing:** HTTP/2 splits application messages into binary frames (HEADERS and DATA) and embeds a Stream ID in each frame's header.
- **Multiplexing:** Interleaves frames from multiple independent streams over a single TCP connection. The receiver reassembles them using Stream IDs, resolving HOL blocking.

[M1.2] DNS Query Hierarchy & Truncation Controls (*DNS Hierarchy*)

- **Query Dual Control:**
- **Recursive Query:** The client requests the Local DNS server to resolve a name. The server must query other servers on behalf of the client and return the final IP.
- **Iterative Query:** The Local DNS server queries Root, TLD, and Authoritative servers in turn, with each level returning the IP of the next server to query.
- **Transport limits:** Uses UDP port 53. If a response exceeds 512 bytes, the server sets the TC (Truncated) bit in the header, forcing the client to reconnect via TCP port 53.

[M1.3] BSD Socket APIs & Kernel Connection Mapping (*net/socket.c*)

- **Server Lifecycle:** `socket()` allocates a file descriptor ☐ `bind()` binds the descriptor to an IP and port ☐ `listen()` transitions the socket to listening and configures half-open (SYN) and established connection queue limits.
- **Accepting Connections:** `accept()` blocks until a connection in the established queue is ready, returning a new socket descriptor for data transfer.
- **Client Connection:** `connect()` initiates the active TCP three-way handshake, setting the local socket state to `SYN_SENT`.

[M1.4] epoll Multiplexing Level & Edge Triggering (*fs/eventpoll.c*)

- **O(1) Event retrieval:** `epoll` avoids `select/poll`'s O(N) scanning of file descriptor lists by tracking sockets in a kernel red-black tree and storing active sockets in a double-linked Ready List.
- **Triggering Modes:**
- **Level Triggered (LT):** `epoll_wait` keeps returning notifications as long as data remains in the socket buffer.
- **Edge Triggered (ET):** Fires a notification only when the buffer state changes. Requires non-blocking sockets and looping reads/writes until returning `EAGAIN` to prevent missing events.

[M2.1] Port Demultiplexing & Socket Binding (*net/ipv4/af_inet.c*)

- **Connectionless Demultiplexing (UDP):** Packets are routed using a 2-tuple (destination IP, destination Port). Packets from different sources targeting the same port enter the same socket buffer.
- **Connection-Oriented Demultiplexing (TCP):** Routes packets using a 4-tuple (source IP, source Port, destination IP, destination Port).
- **Multiple Sockets:** Allows multiple concurrent connections to share the identical destination port on a host, with the kernel hash table routing to distinct sockets.

[M2.2] Go-Back-N (GBN) Sliding Window & Cumulative ACKs (*net/ipv4/tcp_input.c*)

- **Window limit:** Allows the sender to transmit up to N unacknowledged segments, monitored by a single timer for the oldest sent but unacknowledged packet.
- **Cumulative ACK:** The receiver acknowledges only the highest in-order sequence number received (ACK y implies all bytes up to y are received), discarding out-of-order segments.
- **Window Rollback:** On timeout, the sender retransmits all N unacknowledged segments currently in the window, degrading throughput on high-latency paths.

[M2.3] Selective Repeat (SR) Buffering & Lock-Step Alignment (*net/ipv4/tcp_input.c*)

- **Per-Packet Timers:** The sender manages a separate timer for each segment, retransmitting only the specific packet that timed out.
- **Out-of-Order Buffering:** The receiver window has size N (satisfying $N \leq 2^{\text{seq}} - 1$ to avoid sequence number overlap), buffering out-of-order segments and sending individual non-cumulative ACKs.
- **Slide Alignment:** The receiver window slides forward only when the lowest unreceived segment is successfully received and acknowledged.

[M2.4] RTT Estimation & RTO EWMA Calculations (*net/ipv4/tcp_input.c*)

- **Smoothed RTT:** Measures SampleRTT from send to ACK.
- EstimatedRTT $= (1 - \alpha) \times \text{EstimatedRTT} + \alpha \times \text{SampleRTT}$ ($\alpha = 0.125$)
- **Deviation Calculation:**
- DevRTT $= (1 - \beta) \times \text{DevRTT} + \beta \times |\text{SampleRTT} - \text{EstimatedRTT}|$ ($\beta = 0.25$)
- **Timeout Interval:** $\text{TimeoutInterval}(\text{RTO}) = \text{EstimatedRTT} + 4 \times \text{DevRTT}$. The RTO doubles exponentially on successive timeouts (exponential backoff) to reduce congestion.

[M2.5] TCP Connection Handshake & Half-Close FSM (*net/ipv4/tcp_input.c*)

- **Handshake FSM:**
- Client `CLOSED` ☐ sends SYN ☐ `SYN_SENT` ☐ receives SYN-ACK, sends ACK ☐ `ESTABLISHED`.
- Server `LISTEN` ☐ receives SYN, sends SYN-ACK ☐ `SYN_RCVD` ☐ receives ACK ☐ `ESTABLISHED`.
- **Teardown FSM:**
- Active Closer: `ESTABLISHED` ☐ sends FIN ☐ `FIN_WAIT_1` ☐ receives ACK ☐ `FIN_WAIT_2` ☐ receives FIN, sends ACK ☐ `TIME_WAIT` (waits 2MSL to clear stale packets) ☐ `CLOSED`.
- Passive Closer: `ESTABLISHED` ☐ receives FIN, sends ACK ☐ `CLOSE_WAIT` ☐ sends FIN ☐ `LAST_ACK` ☐ receives final ACK ☐ `CLOSED`.

[M3.1] AIMD Additive Increase & ssthresh Thresholds (*net/ipv4/tcp_cong.c*)

- **Slow Start:** $cwnd$ starts at 1 MSS, doubling every RTT (each ACK increments $cwnd$ by 1) until $cwnd \geq ssthresh$.
- **Congestion Avoidance:** $cwnd$ increases linearly by 1 MSS per RTT (Additive Increase).
- **Multiplicative Decrease:** On packet loss (timeout), sets $ssthresh = cwnd/2$, resets $cwnd = 1$ MSS, and enters slow start.

[M3.2] Fast Retransmit & Fast Recovery (FRR / RFC 5681) (*net/ipv4/tcp_input.c*)

- **Triple Duplicate ACKs:** The receiver sends duplicate ACKs on detecting out-of-order segments. If the sender receives 3 duplicate ACKs (4 total), it immediately retransmits the missing segment without waiting for RTO timeout.
- **Fast Recovery:** Avoids resetting $cwnd$ to 1. Sets $ssthresh = cwnd/2$, configures $cwnd = ssthresh + 3 \times \text{MSS}$. On receiving the ACK for the missing segment, sets $cwnd = ssthresh$ and enters Congestion Avoidance.

[M3.3] Cubic Congestion Window growth Curve (*net/ipv4/tcp_cubic.c*)

- **Cubic Window Equation:** Calculates window size $W(t)$ as a cubic function of time t elapsed since the last window reduction:

$$W(t) = C(t - K)^3 + W_{\max}$$

- where $K = \sqrt[3]{W_{\max} \times (1 - \beta)/C}$.
- **Growth Curve behavior:** Window growth slows down significantly as it approaches $W_{\max}$ (the previous congestion point) to probe link capacity gently, then accelerates, ensuring high stability and link utilization.

[M3.4] Google BBR Congestion Control & Kleinrock limits (*net/ipv4/tcp_bbr.c*)

- **Pacing Rate Control:** Unlike loss-based algorithms like Cubic that fill buffers until packet loss occurs (causing bufferbloat), BBR estimates two physical link metrics over a sliding window:
- **Bottleneck Bandwidth (BtlBw):** Max delivery rate.
- **Propagation RTT (RTprop):** Physical delay without queuing.
- **Kleinrock Operating Point:** Keeps the inflight data equal to the Bandwidth-Delay Product (BDP): $\text{pacing_rate} = \text{BtlBw}$ and Inflight $= \text{BtlBw} \times \text{RTprop}$. This maximizes throughput while keeping queue length at zero.

[🔍] Protocol Stack & Physical Transport Trade-offs Matrix

↗️**Unlimited Window:** Increasing congestion window $cwnd$ indefinitely will maximize data throughput.↗️**Router Buffer Overflows:** Excessively large windows cause buffer queues to overflow, triggering massive packet loss and congestion collapse. → **Dual Bound Control:** Limit window size as $W \leq \min(cwnd, rwnd)$, balancing network congestion and receiver limits.

↗️**Aggressive Timeout:** Setting a very small retransmission timeout (RTO) ensures faster recovery on loss.↗️**RTT Jitter & Latency:** Under-estimating RTT leads to spurious retransmissions of delayed packets, wasting network capacity. → **EWMA smoothing + 4*DevRTT variance:** Automatically adjusts RTO based on RTT mean and variance to balance responsiveness and accuracy.

↗️**Loss equals Congestion:** Any packet loss indicates network congestion, requiring window halving.↗️**Wireless Channel Random Noise:** Wireless links suffer from random electromagnetic noise dropouts unrelated to routing congestion. → **BBR Pacing Rate Control:** Regulates transmission using measured bottleneck bandwidth, remaining robust against random packet loss.

↗️**Zero Buffer Ideal:** Buffers on intermediate routing switches should be set to zero to achieve minimal latency.↗️**Traffic Burstiness:** Devoid of queuing space, routers drop packets during any bursty load transitions, degrading TCP throughput. → **Rule of Thumb Buffering:** Size router buffers as $C \times \text{RTT} / \sqrt{N}$ to balance queue delay and throughput for N flows.

[M4.1] IPv4 Fragmentation vs IPv6 Fixed Header (*net/ipv4/ip_input.c*)

- **IPv4 Fragmentation:** Routers segment packets exceeding MTU using Identification (16 bits), Flags (DF, MF), and Fragment Offset (13 bits) fields. Reassembly is performed at the destination host.
- **IPv6 Simplified Header:** Employs a fixed 40-byte header, removing router-level fragmentation (senders must use Path MTU Discovery). Header checksums are removed to speed up processing, and a 20-bit Flow Label is introduced to classify streaming traffic.

[M4.2] Longest Prefix Matching (LPM) via Trie Trees (*net/ipv4/fib_trie.c*)

- **Longest Prefix Match:** Routers must forward packets based on the CIDR table prefix that matches the destination IP address for the longest number of bits.
- **Trie Structures:**
- **Software:** Level-Compressed Trie (LC-Trie) collapses path branches to reduce tree depth, minimizing memory accesses.
- **Hardware:** Core routers use Ternary Content Addressable Memory (TCAM) to match IP bits and wildcard bits (*) in parallel $O(1)$ time.

[M4.3] Crossbar Switch Fabric & Head-of-Line Blocking (*Router Switch*)

- **Crossbar Switch:** Interconnects N input ports to N output ports using intersecting buses, allowing concurrent non-conflicting transfers.
- **Head-of-Line (HOL) Blocking:** Occurs when two packets at different input queues target the same output port, blocking one. The blocked packet at the front of the queue prevents packets behind it from crossing the fabric.
- **Remedy:** Input ports utilize Virtual Output Queueing (VOQ), separating the physical buffer into N logical queues, one per output port.

[M4.4] ICMP Error Reporting & Traceroute Path Discovery (*net/ipv4/icmp.c*)

- **ICMP Diagnostics:** Runs directly over IP (protocol code 1) to send control messages. Type 3 indicates Destination Unreachable; Type 11 indicates Time Exceeded.
- **Traceroute Path Discovery:**
 1. Senders dispatch a UDP packet targeting a high, unused port with TTL=1.
 2. The first-hop router decrements TTL to 0, drops the packet, and returns an ICMP Time Exceeded message (Type 11, Code 0).
 3. The sender records the router's IP, increments TTL (2, 3...), and repeats the process.
 4. The destination host responds with an ICMP Port Unreachable message (Type 3, Code 3), terminating the trace.

[M5.1] Link-State routing (Dijkstra) & OSPF Flooding (*OSPF Daemon*)

- **Global Link-State:** Senders require the complete network topology map to calculate routing paths.
- **OSPF LSA Flooding:** Senders flood Link State Advertisements (LSAs) containing link weights to all other routers in the autonomous system.
- **Shortest Path Trees:** Senders compile a unified Link State Database (LSDB). Once synchronized, each router independently executes Dijkstra's algorithm, producing the shortest-path tree to all destinations.

[M5.2] Distance-Vector Routing (Bellman-Ford) & Count-to-Infinity (*RIP Daemon*)

- **Decentralized Iteration:** Senders use the Bellman-Ford equation: $D_x(y) = \min_v \{ c(x, v) + D_v(y) \}$, periodically exchanging distance vectors with direct neighbors.
- **Count-to-Infinity:** When a link fails, neighbors exchange stale vectors, creating routing loops and causing cost metrics to slowly increment to infinity.
- **Prevention:** Uses Split Horizon (do not advertise routes back to the neighbor from which they were learned) and Poison Reverse (advertise cost as 16 to indicate unreachable).

[M5.3] BGP Path Attributes (AS-Path, Next-Hop, Local-Pref) (*BGP Daemon*)

- **BGP AS Routing:** Manages routing between Autonomous Systems (AS), enforcing routing policies and avoiding loop generation.
- **Core Attributes:**
- **AS-Path:** List of AS numbers through which the route advertisement has passed. If a router detects its own AS in the list, it rejects the advertisement to prevent loops.
- **Next-Hop:** IP address of the boundary router leading to the next AS.
- **Local-Preference:** Configures the policy for outgoing traffic selection within the local AS (higher values are preferred).

[M5.4] BGP Policy Decision Tree Selection (*BGP Engine*)

- **Decision Tree:** If a BGP router receives multiple routes to the same destination prefix, it resolves them using a strict priority hierarchy:
 1. Highest Local Preference (Local-Pref) value.
 2. Shortest AS-Path length.
 3. Lowest Origin type (IGP < EGP < Incomplete).
 4. Lowest Multi-Exit Discriminator (MED) value.
 5. Prefer routes learned via external BGP (eBGP) over internal BGP (iBGP).
 6. Lowest IGP cost to the Next-Hop router.
 7. Lowest BGP Router ID.

[M5.5] Internal BGP (iBGP) vs External BGP (eBGP) routing (*BGP Peering*)

- **External BGP (eBGP):** Session between boundary routers in different ASes to exchange reachability information.
- **Internal BGP (iBGP):** Session between routers inside the same AS to distribute reachability information learned from eBGP.
- **Split-Horizon Rule:** A router cannot advertise routes learned from an iBGP peer to other iBGP peers, requiring a Full Mesh of iBGP connections or Route Reflectors.

[M5.6] Software-Defined Networking (SDN) & OpenFlow Flow Tables (*SDN Architecture*)

- **Control/Data decoupling:** Separates the routing decision logic (control plane, centralized controller) from forwarding switches (data plane).
- **OpenFlow Switch flow tables:** Matches packets using three fields:
 1. **Match Fields:** Matches headers across multiple layers (Port, MAC, IP, TCP/UDP ports).
 2. **Actions:** Execution instructions on match (Forward, Drop, Modify headers, GoTo next table).
 3. **Counters:** Aggregates match statistics for flow monitoring and logging.

[M6.1] Link Framing & CRC Polynomial Calculations (*net/core*)

- **Framing:** Encapsulates IP packets into link-layer frames, wrapping them in headers and trailers.
- **CRC Error Detection:** Senders append R bits calculated over d data bits D .
- A generator polynomial G (size $r + 1$) is selected.
- Computes R such that the combined $(D \cdot 2^r) \oplus R$ is divisible by G modulo 2.
- Receivers perform modulo 2 division of received bits by G . If the remainder is non-zero, the frame is discarded.

[M6.2] Ethernet CSMA/CD Carrier Sense & Exponential Backoff (*drivers/net*)

- **Carrier Sensing:** Senders listen to the channel before transmitting. If busy, they wait; if idle, they transmit.
- **Collision Detection:** Senders monitor the channel while transmitting. If a collision is detected, they abort, send a 48-bit jam signal, and back off.
- **Binary Exponential Backoff:** Senders wait $K \times 512$ bit times before retrying, where K is randomly chosen from $[0, 2^m - 1]$ ($m = \min(n, 10)$ on the n -th collision). Aborts after 16 collisions.

[M6.3] Wi-Fi CSMA/CA Collision Avoidance & RTS/CTS (*net/mac80211*)

- **No Collision Detection:** Wi-Fi radios cannot transmit and listen concurrently.
- **Collision Avoidance:** Senders back off randomly if the channel is busy. If idle, they wait a DIFS duration and send the frame. Senders require an ACK frame from the receiver.
- **RTS/CTS Handshake:** Senders transmit a Request to Send (RTS) frame specifying transmission time. Senders wait for a Clear to Send (CTS) response from the receiver. Neighbors hearing RTS/CTS set their Network Allocation Vector (NAV) to remain silent.

[M6.4] ARP IP-to-MAC Address Resolution & Aging (*net/ipv4/arp.c*)

- **MAC Address Resolution:** Maps IP addresses to MAC addresses for local subnet delivery.
- **Broadcast/Unicast:** Senders broadcast an ARP query frame with destination MAC FF:FF:FF:FF:FF:FF. Senders cache the mapping and assign a Time-To-Live (TTL, e.g., 20 mins) to support interface card changes.

[M6.5] 802.1Q VLAN Tagging & Switch Self-Learning (*net/8021q*)

- **Broadcast Domain Isolation:** Splitting a physical switch into separate logical VLAN domains.
- **802.1Q Framing:** Inserts a 4-byte VLAN tag (containing a 12-bit VLAN ID) into the Ethernet frame header for Trunk links.
- **Switch Self-Learning:** Switches log (Source MAC, Port, VLAN ID) into a MAC table. Senders flood frames on unknown destinations and unicast on cached hits.

🔧 Zone T: top-down-networking Diagnostics & Performance Tuning Reference Laboratory

T1: Kernel Tuning Parameters for Networking

`net.ipv4.tcp_congestion_control`: Configures active congestion control algorithm. Values include cubic and bbr (requires kernel 4.9+ and fq qdisc). `\net.core.rmem_max / \net.core.wmem_max`: Global maximum read/write buffer size limits in bytes. Critical for high Bandwidth-Delay Product (BDP) pipes. `\net.ipv4.tcp_rmem / \net.ipv4.tcp_wmem`: Configures TCP auto-tuning buffer size parameters: `min, default, and max`. `\net.ipv4.tcp_max_syn_backlog`: Maximum size of the half-open connection queue (SYN_RCVD). Tuned higher (with `\net.ipv4.tcp_syncookies` active) to resist SYN flood attacks. `\net.core.somaxconn`: Maximum size of the established connection queue waiting for accept().

T2: Network Troubleshooting & Profiling CLI Commands

`\texttt{\# 1. Capture packet frames and save to pcap format (using tcpdump) \texttt{tcpdump -i eth0 -n -vvv -c 500 -w capture.pcap 'tcp port 80 or udp port 53'}` `\texttt{\# 2. Monitor syn flag connections in real-time (using tshark) \texttt{tshark -i eth0 -Y 'tcp.flags.syn == 1' -T fields -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport}` `\texttt{\# 3. View socket details, cwnd, and RTT estimations (using ss -t) \texttt{ss -t -t -o '\texttt{state, cwnd, ssthresh, rtt, rto, and BBR pacing speeds}'}` `\texttt{\# 4. Perform network hop diagnostics (using mtr / traceroute) \texttt{mtr -r -n -c 10 8.8.8.8}` `\texttt{\# 5. Check interface carrier error stats and collision counts (using ip -s / ethtool) \texttt{ip -s link show eth0}` `\texttt{\# Dumps link-layer frame errors and carrier collisions \texttt{ethtool -S eth0}` `\texttt{\# Queries hardware MAC and driver statistics`